

Recommended OpenWebUI + Ollama Setup for NemoClaw

For persistent NemoClaw and OpenWebUI use, run Ollama as the local inference backend. OpenWebUI will use Ollama as its model backend, and NemoClaw can connect to Ollama through an OpenAI-compatible endpoint if available.

1. Start Ollama

On the host machine:

```
```bash id="2xx817" ollama serve
```

In another terminal, pull or verify your model:

```
```bash id="qq5zf"
ollama pull hermes3:8b
ollama list
```

Ollama's default API endpoint is:

```
```text id="d7flke" http://localhost:11434
```

## 2. Docker Compose example for Ollama + OpenWebUI

```
```yaml id="0cje7e"
services:
  ollama:
    image: ollama/ollama:latest
    container_name: ollama
    restart: unless-stopped
    ports:
      - "11434:11434"
    volumes:
      - ollama-data:/root/.ollama
    deploy:
      resources:
        reservations:
          devices:
            - capabilities: [gpu]
              count: all

  openwebui:
    image: ghcr.io/open-webui/open-webui:main
    container_name: openwebui
```

```

restart: unless-stopped
ports:
  - "3000:8080"
volumes:
  - openwebui-data:/app/backend/data
environment:
  - OLLAMA_BASE_URL=http://ollama:11434
depends_on:
  - ollama

volumes:
  ollama-data:
  openwebui-data:

```

After starting the stack, open OpenWebUI at:

```
``text id="40kp85" http://localhost:3000
```

The Ollama models should appear in OpenWebUI after they are pulled.

3. Test Ollama from the host

```

```bash id="a1yiaf"
curl http://localhost:11434/api/tags

```

Expected result: the response should list installed Ollama models.

Example:

```
```json id="s4gc5m" { "models": [ { "name": "hermes3:8b" } ] }
```

4. Configure OpenWebUI

If OpenWebUI runs in Docker Compose with Ollama in the same stack, use:

```

```text id="84xghn"
Ollama Base URL: http://ollama:11434

```

If OpenWebUI runs on the host and Ollama also runs on the host, use:

```
``text id="gyvabx" Ollama Base URL: http://localhost:11434
```

If OpenWebUI runs in Docker but Ollama runs on the host, use:

```
```text id="2feoq5"
Ollama Base URL: http://host.docker.internal:11434
```

On Linux, you may need to add this to the OpenWebUI service:

```
```yaml id="jwnnjc" extra_hosts: - "host.docker.internal:host-gateway"
```

### ## 5. Onboard NemoClaw against Ollama

If NemoClaw supports Ollama directly, use the Ollama provider during onboarding.

Example non-interactive shape:

```
```bash id="fz9v8x"
NEMOCLAW_PROVIDER=ollama
NEMOCLAW_ENDPOINT_URL=http://localhost:11434
NEMOCLAW_MODEL=hermes3:8b
nemoclaw onboard --non-interactive
```

If NemoClaw expects an OpenAI-compatible endpoint, use Ollama's OpenAI-compatible API path:

```
```bash id="1jcidg" NEMOCLAW_PROVIDER=custom
NEMOCLAW_ENDPOINT_URL=http://localhost:11434/v1
NEMOCLAW_MODEL=hermes3:8b
COMPATIBLE_API_KEY=dummy
nemoclaw onboard --non-interactive
```

Use a non-empty placeholder API key, such as:

```
```bash id="6t6zmj"
COMPATIBLE_API_KEY=dummy
```

because some OpenAI-compatible clients require an API key string even when the local endpoint does not enforce authentication.

6. Recommended architecture

```
```text id="qfx5ot" OpenWebUI -> Ollama -> Hermes 3 / local Ollama model
```

```
NemoClaw -> Ollama directly -> same local Ollama model ```
```

This keeps OpenWebUI and NemoClaw independent while sharing the same local inference backend.

## 7. Notes

Ollama is easier to run than vLLM, but tool calling behavior may depend on the model and client compatibility.

For NemoClaw-style agent/tool workflows, prefer a model that supports tool/function calling well.

If OpenWebUI can chat with the model but NemoClaw tool calls fail, check whether the selected Ollama model supports tool calling and whether NemoClaw is using the native Ollama endpoint or the OpenAI-compatible `/v1` endpoint.

If the model does not behave correctly with tools, try another Ollama model with stronger tool-calling support.